

EngiSense AI

AI-Powered Engineering Intelligence Platform

Full-Stack Web Application with
Data Analytics, Generative AI and Retrieval-Augmented Generation

React + FastAPI + PostgreSQL
LangChain + Hugging Face + RAG
Pandas + NumPy + Matplotlib + Seaborn
Docker + Git + GitHub

Project Type: Full-Stack AI / Data Analytics Platform
Domain: Engineering Data Intelligence
Development Model: Modular and Iterative
Version: 1.0

Project Proposal and Technical Documentation

Contents

1	Project Overview	3
1.1	Project Title	3
1.2	Project Concept	3
1.3	Core Idea	3
2	Problem Statement	3
3	Project Objectives	4
4	Target Users	5
5	Key Features	5
5.1	User Authentication	5
5.2	Project Management	5
5.3	Dataset Management	6
5.4	Data Analysis	6
5.5	Data Visualization	7
5.6	Document Intelligence	7
5.7	Retrieval-Augmented Generation	7
5.8	AI Assistant	8
5.9	Data-Aware AI	8
6	System Architecture	8
7	Technology Stack	9
8	Software Architecture	10
9	Data Analysis Pipeline	11
10	Generative AI Architecture	12
11	Retrieval-Augmented Generation Pipeline	12
12	Database Design	13
13	API Design	14
14	Docker Architecture	14
15	Version Control and GitHub Strategy	15
16	Security Considerations	15
17	Testing Strategy	16
17.1	Backend Testing	16
17.2	Frontend Testing	16
17.3	AI Testing	16

18 CI/CD Strategy	17
19 Development Phases	17
19.1 Phase 0 – Requirements and Planning	17
19.2 Phase 1 – System Architecture	17
19.3 Phase 2 – Backend Foundation	18
19.4 Phase 3 – Authentication and Projects	18
19.5 Phase 4 – Dataset Analysis	18
19.6 Phase 5 – React Frontend	18
19.7 Phase 6 – Document Intelligence	19
19.8 Phase 7 – RAG and GenAI	19
19.9 Phase 8 – Data-Aware AI	19
19.10Phase 9 – Dockerization	19
19.11Phase 10 – Testing and CI/CD	19
19.12Phase 11 – Deployment	20
20 Expected Final Product	20
21 Future Enhancements	20
22 Project Outcome	21
23 Conclusion	22

1 Project Overview

1.1 Project Title

EngiSense AI – AI-Powered Engineering Intelligence Platform

1.2 Project Concept

EngiSense AI is a full-stack engineering intelligence platform designed to combine data analytics, document intelligence, Generative AI, and modern web technologies into a single application.

The platform allows users to upload engineering datasets and technical documents, analyze structured data, generate interactive visualizations, retrieve relevant information from documents, and interact with an AI assistant capable of answering questions using both structured and unstructured information.

The project combines software engineering, data analysis, artificial intelligence, and engineering-domain applications in a single modular system.

1.3 Core Idea

The central idea of the project is:

“Upload engineering data and documents, analyze the information, visualize important patterns, and interact with an AI assistant that can answer questions using the available engineering knowledge and data.”

The platform is designed to demonstrate how modern technologies can be integrated into a production-oriented software system rather than being implemented as isolated demonstrations.

2 Problem Statement

Engineering teams and students frequently work with multiple sources of information, including:

- CSV and spreadsheet datasets
- Sensor measurements
- Experimental results
- Equipment logs
- Technical reports
- Research documents
- Test results
- Maintenance records

These sources are often analyzed using separate tools. Numerical analysis may be performed using Python, documents may be searched manually, and visualization may require another application.

This fragmented workflow makes it difficult to obtain a unified understanding of the available information.

EngiSense AI aims to address this problem by providing a centralized platform capable of:

1. Managing engineering datasets.
2. Performing statistical and numerical analysis.
3. Generating visualizations.
4. Processing technical documents.
5. Retrieving relevant information from documents.
6. Providing an AI-powered conversational interface.
7. Combining structured data analysis with unstructured document retrieval.

3 Project Objectives

The major objectives of EngiSense AI are:

1. Develop a modern full-stack web application using React and FastAPI.
2. Implement PostgreSQL as the primary relational database.
3. Build an engineering data-analysis pipeline using Pandas and NumPy.
4. Generate engineering-oriented visualizations using Matplotlib and Seaborn.
5. Implement a document-processing and Retrieval-Augmented Generation (RAG) pipeline.
6. Integrate open-source Large Language Models available through Hugging Face.
7. Use LangChain for LLM orchestration, retrieval and tool integration.
8. Enable users to interact with datasets and documents using natural language.
9. Containerize the application using Docker.
10. Maintain the complete project using Git and GitHub.
11. Implement software engineering practices such as modular architecture, testing, environment configuration, logging, documentation and CI/CD.

4 Target Users

The initial target users include:

- Engineering students
- Researchers
- Electronics and embedded engineers
- Data analysts
- Software developers working with engineering data
- Academic laboratories
- Engineering project teams

The system will initially focus on engineering-oriented datasets, but the architecture will be designed to remain domain-independent where practical.

5 Key Features

5.1 User Authentication

The platform will provide user authentication and authorization.

Potential functionality includes:

- User registration
- User login
- Password hashing
- Token-based authentication
- Protected API endpoints
- User-specific projects and datasets

5.2 Project Management

Users will be able to create projects that organize their datasets and documents.

Example:

Project: EV Battery Analysis

```
battery_data.csv
test_report.pdf
maintenance_report.pdf
analysis results
```

5.3 Dataset Management

Users will be able to upload and manage datasets.

Supported initial format:

- CSV

Future support may include:

- Excel
- JSON
- Database sources

5.4 Data Analysis

The system will analyze uploaded datasets using:

- Pandas
- NumPy
- Matplotlib
- Seaborn

Potential analysis operations include:

- Dataset statistics
- Missing-value analysis
- Mean, median and standard deviation
- Minimum and maximum values
- Correlation analysis
- Distribution analysis
- Time-series analysis
- Outlier detection
- Basic anomaly identification

5.5 Data Visualization

The system will automatically generate visualizations based on the dataset.

Examples include:

- Line plots
- Bar charts
- Histograms
- Scatter plots
- Box plots
- Correlation heatmaps

5.6 Document Intelligence

Users will be able to upload technical documents.

The system will process documents through a document-processing pipeline.

The initial document types may include:

- PDF
- TXT
- DOCX

5.7 Retrieval-Augmented Generation

The platform will implement RAG to allow the AI assistant to retrieve relevant information from uploaded documents before generating a response.

The basic pipeline is:

```
Document
  |
  v
Text Extraction
  |
  v
Text Chunking
  |
  v
Embeddings
  |
  v
Vector Store
  |
  v
Retriever
  |
  v
```

Language Model v Generated Response
--

5.8 AI Assistant

The platform will contain a conversational AI assistant.

Example user query:

“What was the maximum operating temperature mentioned in the battery test report?”

The system will retrieve relevant information and generate an answer using the configured Hugging Face model.

5.9 Data-Aware AI

An important feature of the platform is the ability to combine AI with numerical data analysis.

For example:

“Why did the temperature increase significantly during cycle 120?”

The system may use:

1. Dataset analysis tools.
2. Statistical calculations.
3. Relevant technical documents.
4. The language model.

This allows the AI assistant to work with actual project information instead of relying only on the model’s pretrained knowledge.

6 System Architecture

The overall architecture consists of several major layers.

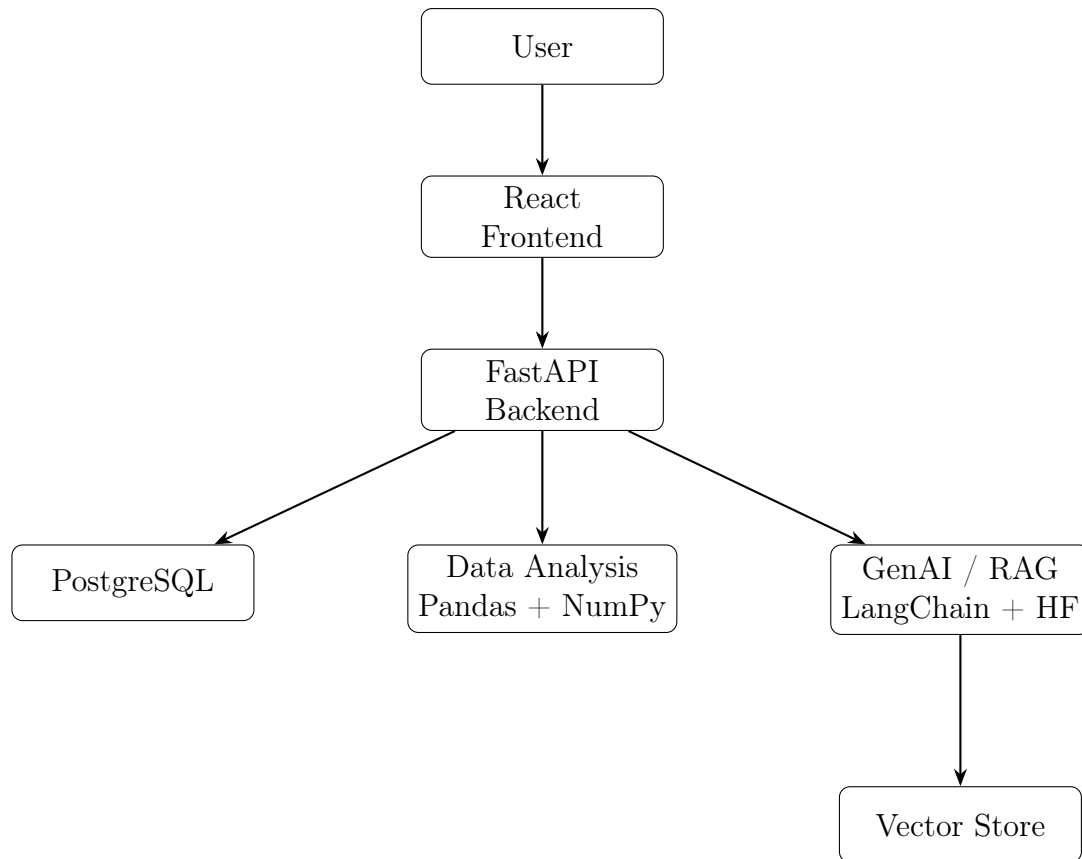


Figure 1: High-Level System Architecture

7 Technology Stack

Category		Technology
Frontend		React
Frontend Styling		CSS / Tailwind CSS
Backend		FastAPI
Programming	Language	Python
Database		PostgreSQL
ORM		SQLAlchemy
Data Validation		Pydantic
Database Migration		Alembic
Data Analysis		Pandas, NumPy
Visualization		Matplotlib, Seaborn
Generative AI		Hugging Face Open Models
LLM Orchestration		LangChain
RAG		LangChain-based RAG Pipeline
Embeddings		Hugging Face Embedding Models
Vector Storage		PostgreSQL + pgvector or suitable vector store
Authentication		JWT-based authentication
Testing		Pytest

API Testing	Postman / OpenAPI
Containerization	Docker
Multi-container Management	Docker Compose
Version Control	Git
Repository Hosting	GitHub
CI/CD	GitHub Actions
Reverse Proxy	Nginx
Documentation	Markdown / OpenAPI

8 Software Architecture

The backend will follow a modular architecture rather than placing all functionality inside a single application file.

A possible backend structure is:

```
backend/
|
+-- app/
| |
| | +-- api/
| | | +-- routes/
| | |
| | +-- models/
| | |
| | +-- schemas/
| | |
| | +-- services/
| | |
| | +-- analysis/
| | |
| | +-- ai/
| | |
| | +-- core/
| | |
| | +-- database/
| | |
| | +-- main.py
|
+-- tests/
|
+-- requirements.txt
|
+-- Dockerfile
```

The frontend will follow a component-based architecture.

```
frontend/
|
+-- src/
| |
```

```
| +-- components/
| +-- pages/
| +-- layouts/
| +-- services/
| +-- hooks/
| +-- utils/
| +-- assets/
| +-- App.jsx
| +-- main.jsx
|
+-- package.json
|
+-- Dockerfile
```

9 Data Analysis Pipeline

The dataset analysis pipeline will follow the following workflow:

```
Dataset Upload
  |
  v
File Validation
  |
  v
Data Loading
  |
  v
Pandas DataFrame
  |
  +-----+
  | |
  v v
Data Cleaning Statistical Analysis
  | |
  +-----+-----+
                |
                v
            Feature Analysis
                |
                v
            Visualization
                |
                v
            Analysis Results
```

The analysis service will be designed independently from the frontend so that the same analysis functionality can be accessed through APIs or AI tools.

10 Generative AI Architecture

The Generative AI component will use open-source models available through Hugging Face.

LangChain will be used as the orchestration layer.

The high-level architecture is:

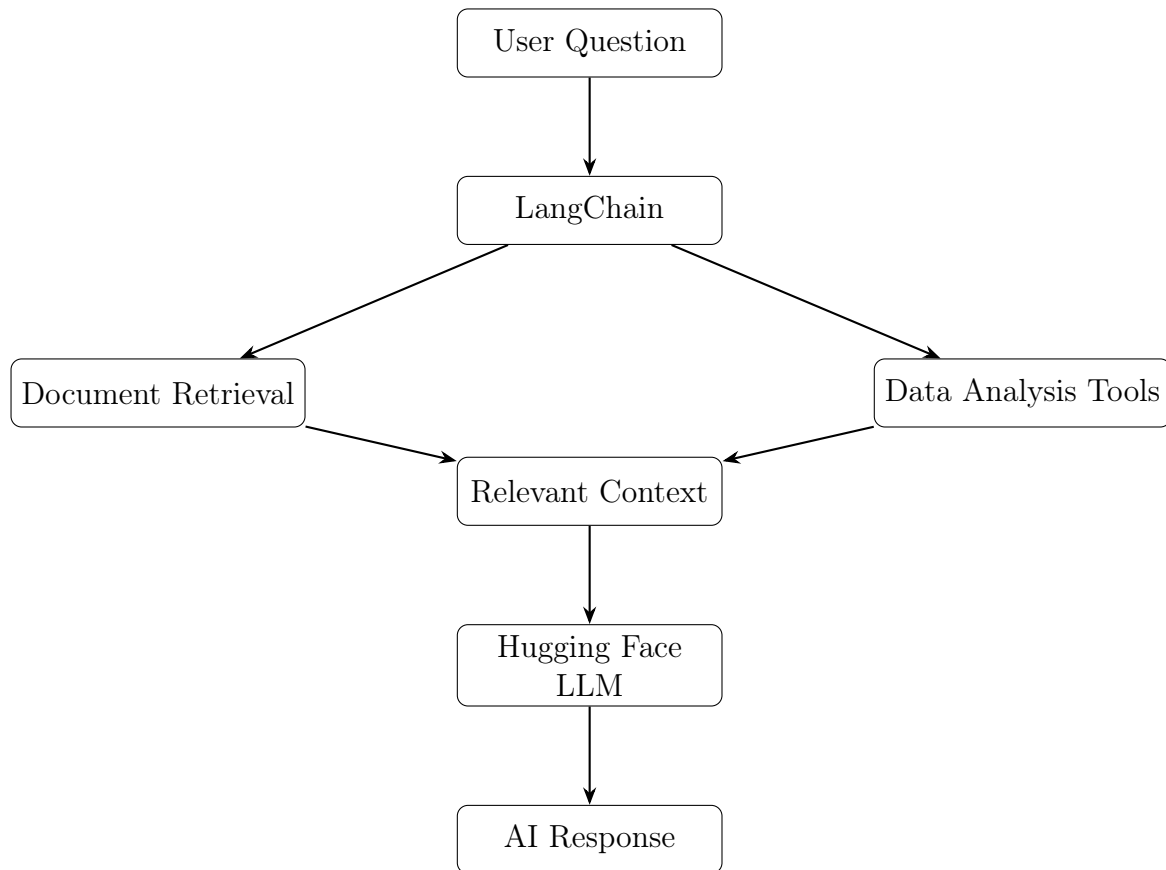


Figure 2: GenAI and RAG Architecture

11 Retrieval-Augmented Generation Pipeline

The RAG subsystem will convert uploaded documents into searchable knowledge.

1. Document upload.
2. Text extraction.
3. Document cleaning.
4. Text chunking.
5. Embedding generation.
6. Vector storage.
7. Similarity search.

8. Retrieval of relevant chunks.
9. Context construction.
10. LLM generation.
11. Response returned to the user.

The purpose of this architecture is to reduce dependence on the pretrained knowledge of the LLM and allow the system to answer questions using user-provided information.

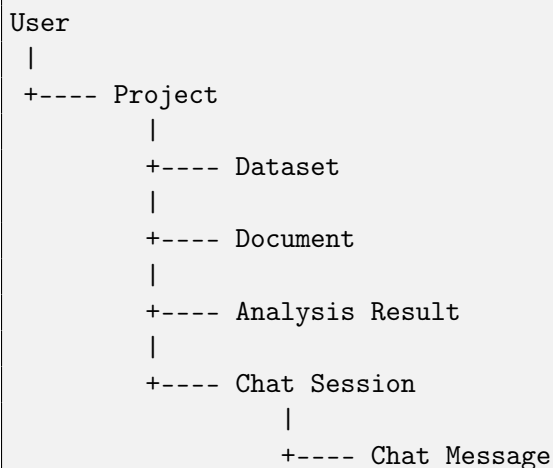
12 Database Design

PostgreSQL will be used as the primary relational database.

Initial entities may include:

- Users
- Projects
- Datasets
- Documents
- Analysis Results
- Chat Sessions
- Chat Messages

A simplified relationship is:



The exact database schema will be finalized during the database-design phase.

13 API Design

FastAPI will expose RESTful endpoints for communication between the frontend and backend.

Example endpoints include:

Method	Endpoint	Purpose
POST	/auth/register	Register user
POST	/auth/login	Authenticate user
GET	/projects	Retrieve projects
POST	/projects	Create project
POST	/datasets/upload	Upload dataset
GET	/datasets	Retrieve datasets
GET	/datasets/{id}	Retrieve dataset information
POST	/analysis/{id}	Analyze dataset
POST	/documents/upload	Upload document
POST	/chat	Send AI query
GET	/chat/history	Retrieve chat history

The API design will be expanded during implementation.

14 Docker Architecture

The application will be containerized to provide consistent development and deployment environments.

The initial Docker architecture is:

```
Docker Compose
|
+-- frontend
| |
| +-- React
|
+-- backend
| |
| +-- FastAPI
| +-- Python
| +-- AI Services
| +-- Data Analysis
|
+-- postgres
|
+-- PostgreSQL
```

Docker Compose will be used to manage the local multi-container environment. Additional services will only be introduced if required by the final architecture.

15 Version Control and GitHub Strategy

Git will be used for source-code version control and GitHub will host the project repository.

The project will follow a feature-based development workflow.

Example:

```
main
|
+-- develop
|
+-- feature/authentication
|
+-- feature/dataset-analysis
|
+-- feature/document-processing
|
+-- feature/rag
|
+-- feature/ai-chat
|
+-- feature/dashboard
```

Each feature will be developed independently and integrated after testing.
The repository will also contain:

- README documentation
- Setup instructions
- API documentation
- Environment configuration example
- Docker configuration
- Testing instructions
- Architecture documentation

16 Security Considerations

Security will be considered throughout development.

Initial security requirements include:

- Password hashing.
- JWT-based authentication.
- Protected API endpoints.
- Input validation.

- File-type validation.
- File-size restrictions.
- Environment variables for secrets.
- Secure database credentials.
- Avoiding hard-coded API keys.
- Basic API error handling.

Sensitive configuration values will not be committed to GitHub.

An example configuration file will be provided:

```
.env.example

DATABASE_URL=
SECRET_KEY=
HF_API_KEY=
MODEL_NAME=
```

17 Testing Strategy

Testing will be introduced throughout the development process.

17.1 Backend Testing

The backend will use Pytest for:

- Unit tests
- Service tests
- API tests
- Authentication tests

17.2 Frontend Testing

Frontend components and important user workflows will be tested as the application develops.

17.3 AI Testing

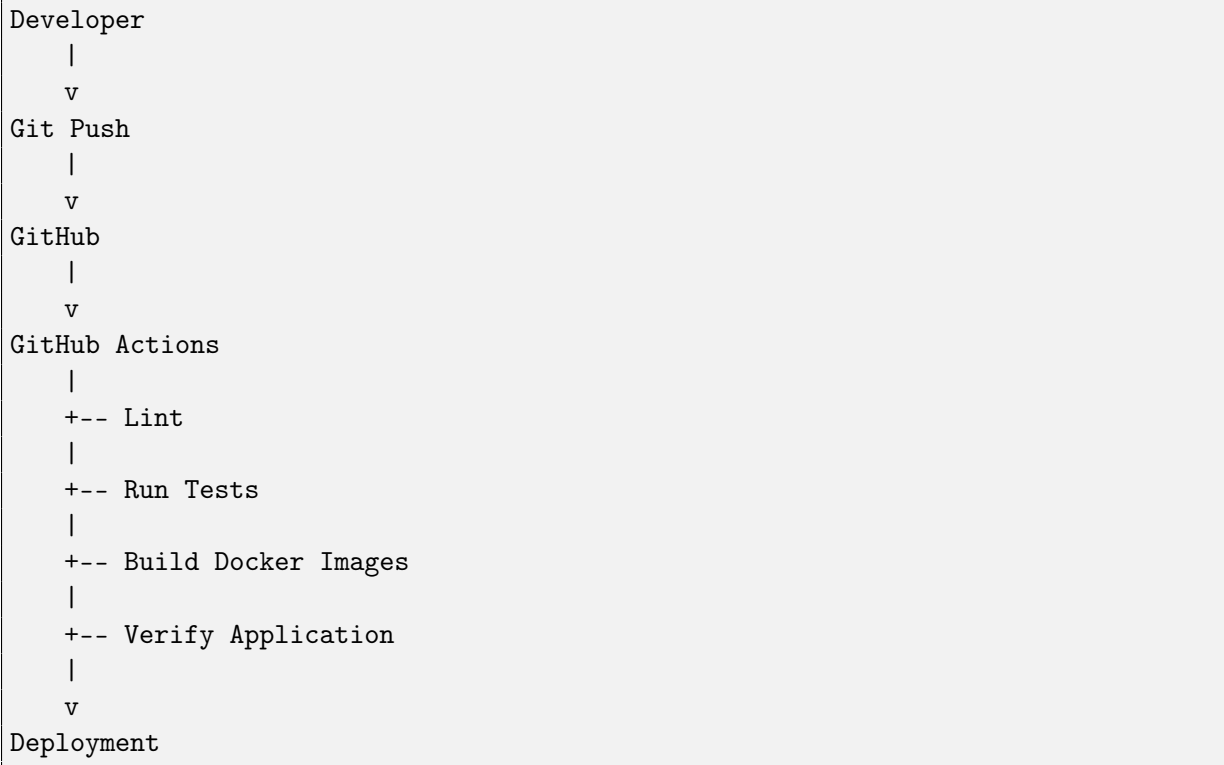
The AI subsystem will be evaluated for:

- Retrieval accuracy
- Response relevance
- Hallucination behavior
- Context usage
- Tool selection

18 CI/CD Strategy

GitHub Actions will eventually be used to automate development checks.

A simplified workflow is:



CI/CD will be introduced after the core application becomes stable.

19 Development Phases

The project will be developed incrementally.

19.1 Phase 0 – Requirements and Planning

- Finalize project scope.
- Define users and use cases.
- Define MVP.
- Define functional requirements.
- Define non-functional requirements.

19.2 Phase 1 – System Architecture

- Design application architecture.
- Design database schema.
- Define API structure.

- Define frontend architecture.
- Define AI architecture.

19.3 Phase 2 – Backend Foundation

- Configure FastAPI.
- Configure PostgreSQL.
- Configure SQLAlchemy.
- Implement database models.
- Configure Alembic.
- Implement basic APIs.

19.4 Phase 3 – Authentication and Projects

- User registration.
- Login.
- Authentication.
- Project management.

19.5 Phase 4 – Dataset Analysis

- Dataset upload.
- CSV validation.
- Pandas processing.
- NumPy analysis.
- Statistical analysis.
- Visualization generation.

19.6 Phase 5 – React Frontend

- Application layout.
- Authentication UI.
- Dashboard.
- Dataset interface.
- Visualization interface.

19.7 Phase 6 – Document Intelligence

- Document upload.
- Text extraction.
- Chunking.
- Embeddings.
- Vector storage.

19.8 Phase 7 – RAG and GenAI

- Integrate LangChain.
- Integrate Hugging Face model.
- Implement retrieval.
- Implement RAG pipeline.
- Build AI chat interface.

19.9 Phase 8 – Data-Aware AI

- Create data-analysis tools.
- Connect tools with LangChain.
- Enable AI to use dataset statistics.
- Implement combined document and dataset reasoning.

19.10 Phase 9 – Dockerization

- Create Dockerfiles.
- Create Docker Compose configuration.
- Configure services.
- Test complete containerized application.

19.11 Phase 10 – Testing and CI/CD

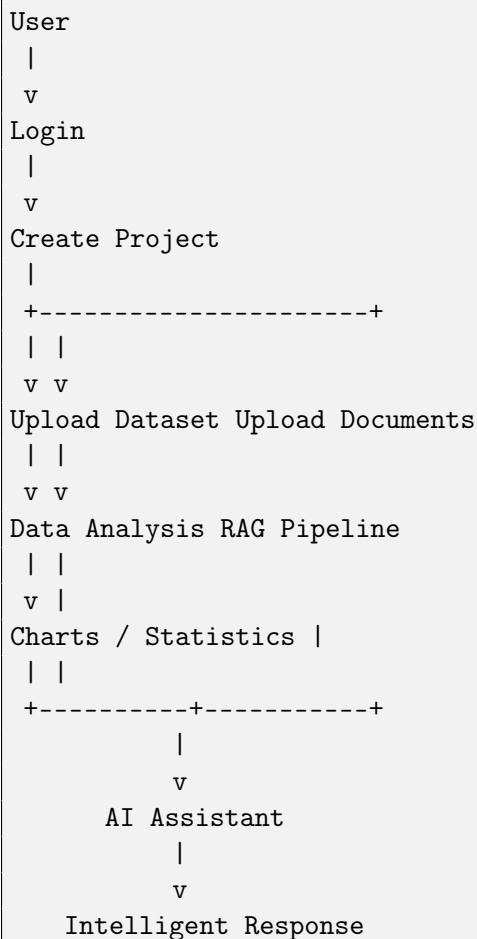
- Add automated tests.
- Configure GitHub Actions.
- Add linting and validation.
- Build Docker images automatically.

19.12 Phase 11 – Deployment

- Select deployment environment.
- Deploy backend.
- Deploy frontend.
- Configure database.
- Configure environment variables.
- Configure monitoring and logging.

20 Expected Final Product

The completed platform is expected to provide the following workflow:



The final application will provide a unified interface for engineering data analysis and AI-assisted information retrieval.

21 Future Enhancements

Potential future enhancements include:

- Real-time IoT data ingestion.
- ESP32 sensor integration.
- Live telemetry dashboards.
- Streaming data analysis.
- Advanced anomaly detection.
- Predictive maintenance.
- Time-series forecasting.
- Role-based access control.
- Team collaboration.
- Advanced vector search.
- Model fine-tuning.
- Cloud deployment.
- Kubernetes-based deployment.
- Monitoring and observability.

These features will not be included in the initial MVP unless they become necessary for the core product.

22 Project Outcome

The primary outcome of this project will be a complete full-stack AI application demonstrating the integration of:

- Modern frontend development.
- REST API development.
- Relational database management.
- Data analysis.
- Data visualization.
- Generative AI.
- Retrieval-Augmented Generation.
- Open-source language models.
- Containerized application development.
- Version control.

- Automated testing.
- CI/CD.

The project is intended to demonstrate not only individual technologies but also the ability to design and develop a complete software system from requirements through deployment.

23 Conclusion

EngiSense AI is proposed as a modular engineering intelligence platform combining full-stack development, data analytics and Generative AI.

The project provides a practical environment for implementing React, FastAPI, PostgreSQL, Python data-analysis libraries, LangChain, Hugging Face models, RAG, Docker and GitHub within one coherent architecture.

The development approach will prioritize a functional MVP first, followed by progressively advanced AI, analytics, deployment and engineering capabilities.

The final system will therefore serve both as a functional application and as a demonstration of modern software engineering and AI application-development practices.